

2.1 - CNNs IN COMPUTER VISION

CHAPTER INDEX

- ☐ [COMPUTER VISION TASKS](#)
- ☐ [DETECTION](#)
 - [REGION BASED CNN](#)
 - [FAST R-CNN](#)
 - [YOLO](#)
 - [BEYOND YOLO V1](#)
- ☐ [SEGMENTATION](#)
 - [DEEPLAB V3+](#)
 - [LOSS FUNCTIONS FOR SEGMENTATION](#)

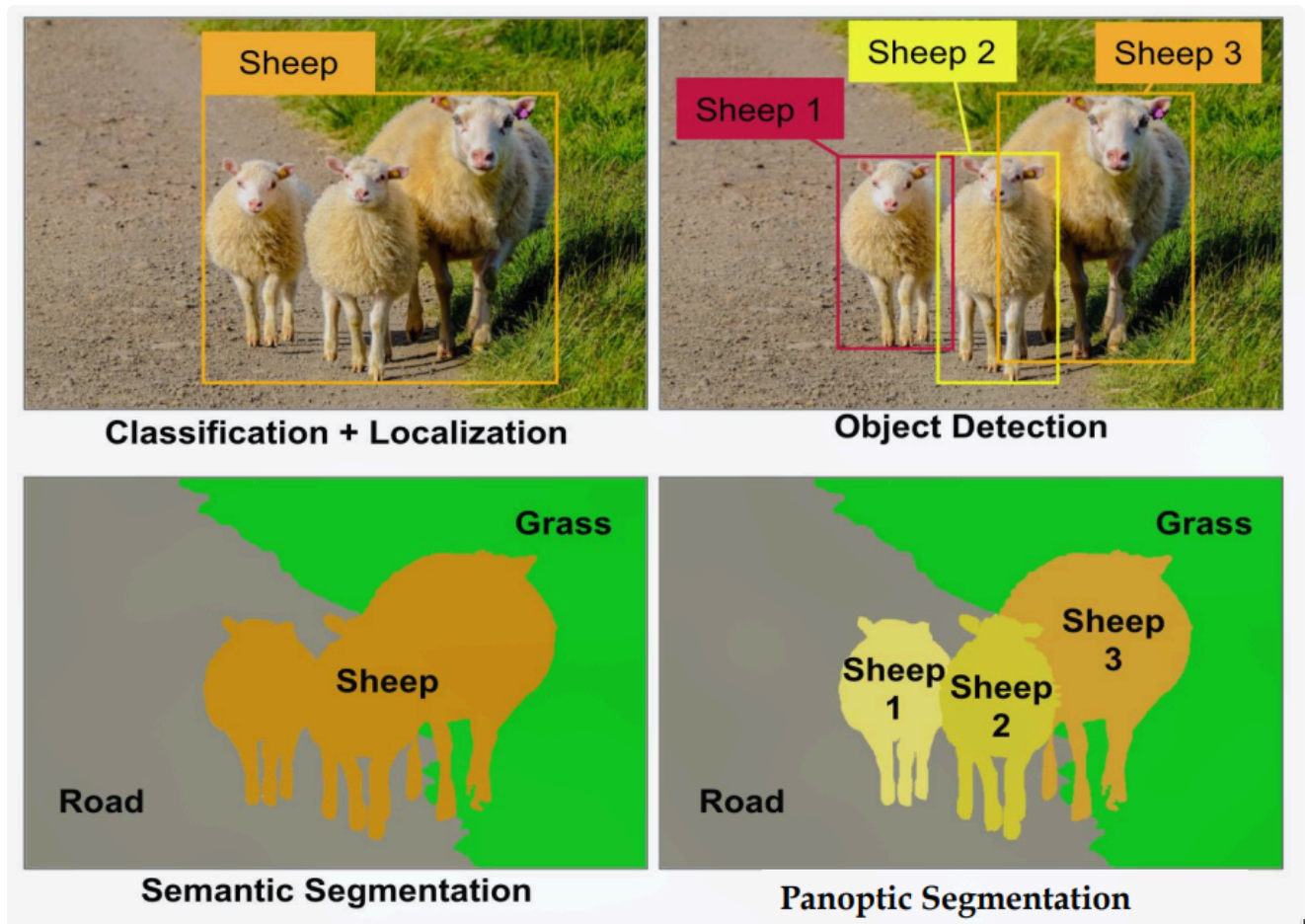
COMPUTER VISION TASKS

Now that we know how CNNs work (see > [2.0 - CONVOLUTIONAL NEURAL NETWORKS](#)), we can take a look at how they can be used in the field of computer vision to tackle image analysis problems.

Before diving into the details, it is worth introducing the **main problems** that computer vision deals with; particularly, computer vision tasks can be organised according to a spectrum of **increasing spatial granularity**.

As we move from **classifying an entire image** to **localizing objects** and finally to **labeling individual pixels**, the complexity of the output and the required architectural design increase accordingly.

Task	Input	Output	Granularity
Classification	Image	One label per image	Whole image
Object Detection	Image	Bounding boxes + class labels for each object	Object-level
Semantic Segmentation	Image	Class label for every pixel (ignoring object instances)	Pixel-level (category-aware)
Instance Segmentation	Image	Class label + instance ID for every pixel	Pixel-level (instance-aware)



This chapter covers the two most advanced tasks on this spectrum: **Object Detection** (localizing objects via boxes) and **Segmentation** (predicting pixel-wise masks). We will explore how **Convolutional Neural Networks** are adapted for each, from region-proposal methods to fully convolutional architectures.

DETECTION

As introduced above, object detection consists in **identifying instances of objects within an image** and in **classifying them** as members of a certain class.

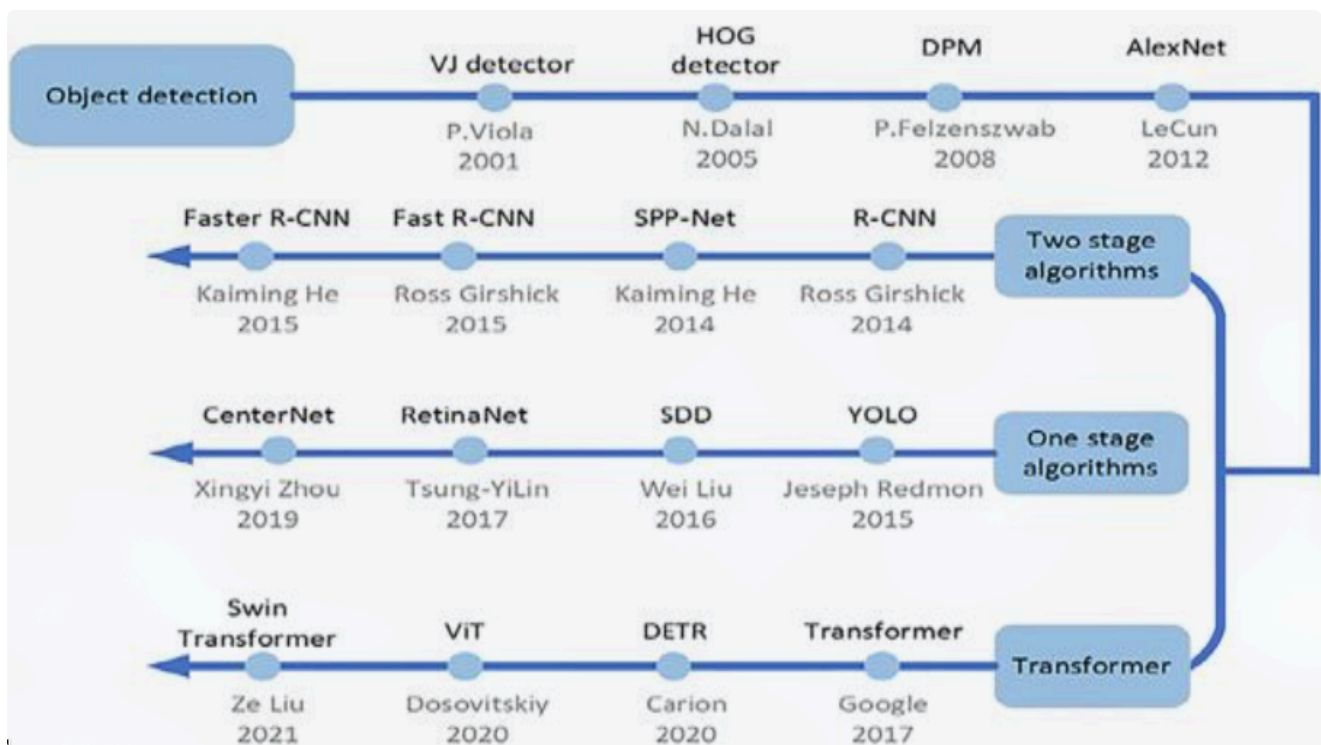
This means we're trying to answer the question:

What objects are in the image, and where?

Current models for object detection can be divided into **two main categories**:

- **Two-stage detectors.** These break the problem into two phases: first, they generate **region proposals**, then they **classify them**.
- **One-stage detectors.** These **directly draw bounding boxes** and apply class labels in a **single pass**.

The following paragraphs focus on **pioneer models of both categories**: R-CNNs for the two-stage detectors, and **YOLO** for the one-stage detectors.



NOTE

The **transformer architecture** will be discussed later in a dedicated chapter due to its importance in the development of **modern LLMs** (see > [3.1 - TRANSFORMERS](#)).

REGION BASED CNN

Region based CNNs, also known as **R-CNNs**, are **two-stage models** developed starting in 2014.

Their approach to the **detection problem** may seem **naive** at first, especially by *modern standards* (though it is important to remember that some of the techniques employed, like SVMs, were considered state of the art at the time), but they represented **one big step** in the **application of deep learning to object detection**.

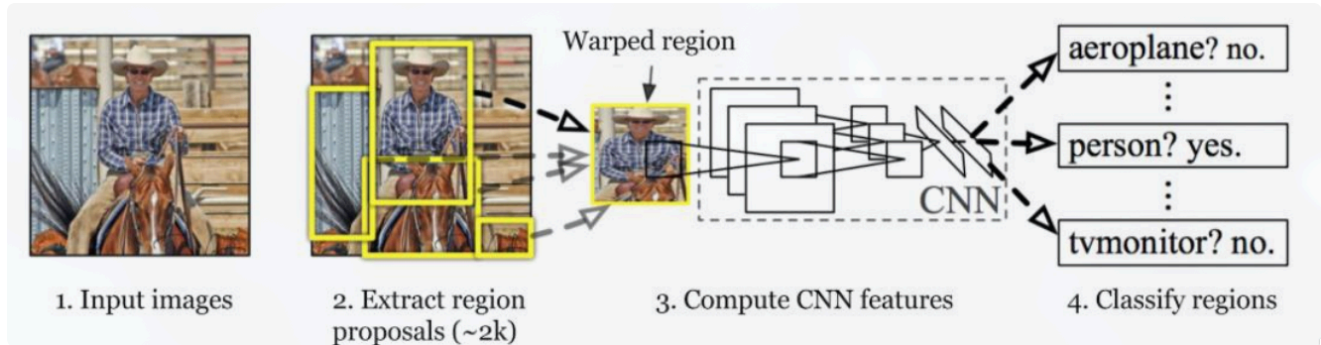
An **R-CNN** follows the **steps** below to tackle the detection problem:

R-CNN SOLUTION

1. **Guess 2000** (this number may vary) **places** where an **object** might be: these are called **region proposals**.
2. Feed **each region** proposal to a **pretrained CNN** to perform **feature extraction**.
3. Feed the **extracted features** to a **set of SVMs** (Support Vector Machines) to obtain the **class of each region**.

4. Use a **linear regression model** to perform the **bounding box regression** (refining the region proposals to enclose objects more accurately).

The figure below illustrates the above mentioned steps:



Now let us go into more details about each step.

STEP 1: Region Proposals

This step relies on the **selective search algorithm**, which works as follows:

1 - Graph-Based Image Segmentation

Segment image into initial regions $R = \{r_1, \dots, r_n\}$ using Felzenszwalb and Huttenlocher (2004)

2 - Setup

Initialise similarity set $S = \emptyset$

foreach Neighbouring region pair (r_i, r_j) do

 Calculate similarity $s(r_i, r_j)$

$S = S \cup s(r_i, r_j)$

3 - Region merging

while $S \neq \emptyset$ do

 Get highest similarity $s(r_i, r_j) = \max(S)$

 Merge corresponding regions $r_t = r_i \cup r_j$

 Remove similarities regarding r_i : $S = S \setminus s(r_i, r^*)$

 Remove similarities regarding r_j : $S = S \setminus s(r^*, r_j)$

 Calculate similarity set S_t between r_t and its neighbours

$S = S \cup S_t$

$R = R \cup r_t$

4 - Proposals

Extract object location boxes L from all regions in R

Let us go through the algorithm in detail:

📋 SELECTIVE SEARCH

1. A **graph-based segmentation method** is used to break the image down into **small regions** called *superpixels*, which are used as **building blocks for the region proposals**.
2. **Similarities** between **neighbouring regions** are calculated to obtain the *similarity set*.
3. A loop keeps **merging regions** with the **highest similarity** as long as the similarity set is non-empty. To guarantee correctness, **similarities are updated** to account for the *merged* regions. Furthermore, **each merged region** is added to a set R .
4. **Object location boxes** are extracted from R .

💡 OBSERVATION

The set R contains **merged regions of varying sizes**, since merges can happen both at the superpixel level *and* at the whole image level (when merging regions that themselves resulted from *previous merges*).

This guarantees that the region proposals can **capture objects at different scales and aspect ratios**.

STEP 2: Feature Extraction

A **pretrained CNN** (e.g. on the ImageNet dataset) is used to perform **feature extraction on each region proposal**.

✍️ NOTE

Because we are simply interested in **extracting features** from each proposed region, we can simply **use the output** of one of the **internal layers** of the CNN and **truncate the ones after**.

Usually, the **last hidden layer** output is chosen, thus truncating just the output layer.

Since CNNs expect **inputs of fixed size**, **each region** proposal must be **resized** to the **correct input size** required by the chosen architecture.

The resized regions are then **fed to the CNN**, which outputs the desired **feature vector**.

STEP 3: Classifying the regions

After obtaining a **feature vector** for **each region** proposal, we need to **classify them**. To do so, the authors of the R-CNN paper trained a **set of binary SVMs** (one for each

object class).

During inference, each region proposal's feature vector is **passed through all N SVMs**, each **outputting a confidence score**. These scores indicate **how confident** the model is that the region proposal **contains a specific object class**.

✍ WHY SVMs OVER THE SOFTMAX LAYER

The authors found **SVMs to be more precise** than the network's own softmax output layer.

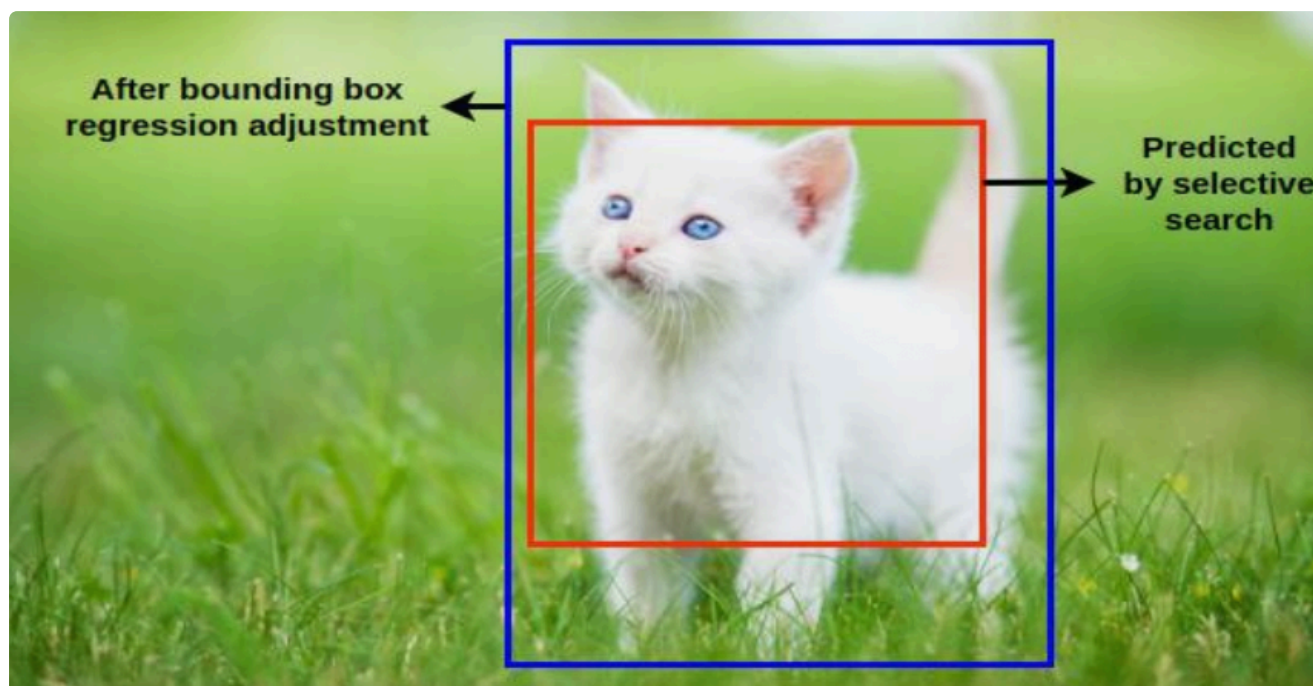
Though this might seem strange now, we need to remember that **SVMs were considered state of the art at the time**.

STEP 4: Bounding Box Regression

The **region proposals** from *Selective Search* are good, but they are **rarely perfect**.

They often **cut off parts** of the object or **include too much** surrounding background.

Bounding Box Regression learns a **transformation** that **maps the proposal box** (P_x, P_y, P_w, P_h) (Notice that a block is described by the **coordinates** of its **centre** and by its **dimensions**) to a tighter, more **accurate ground-truth box** (G_x, G_y, G_w, G_h) .



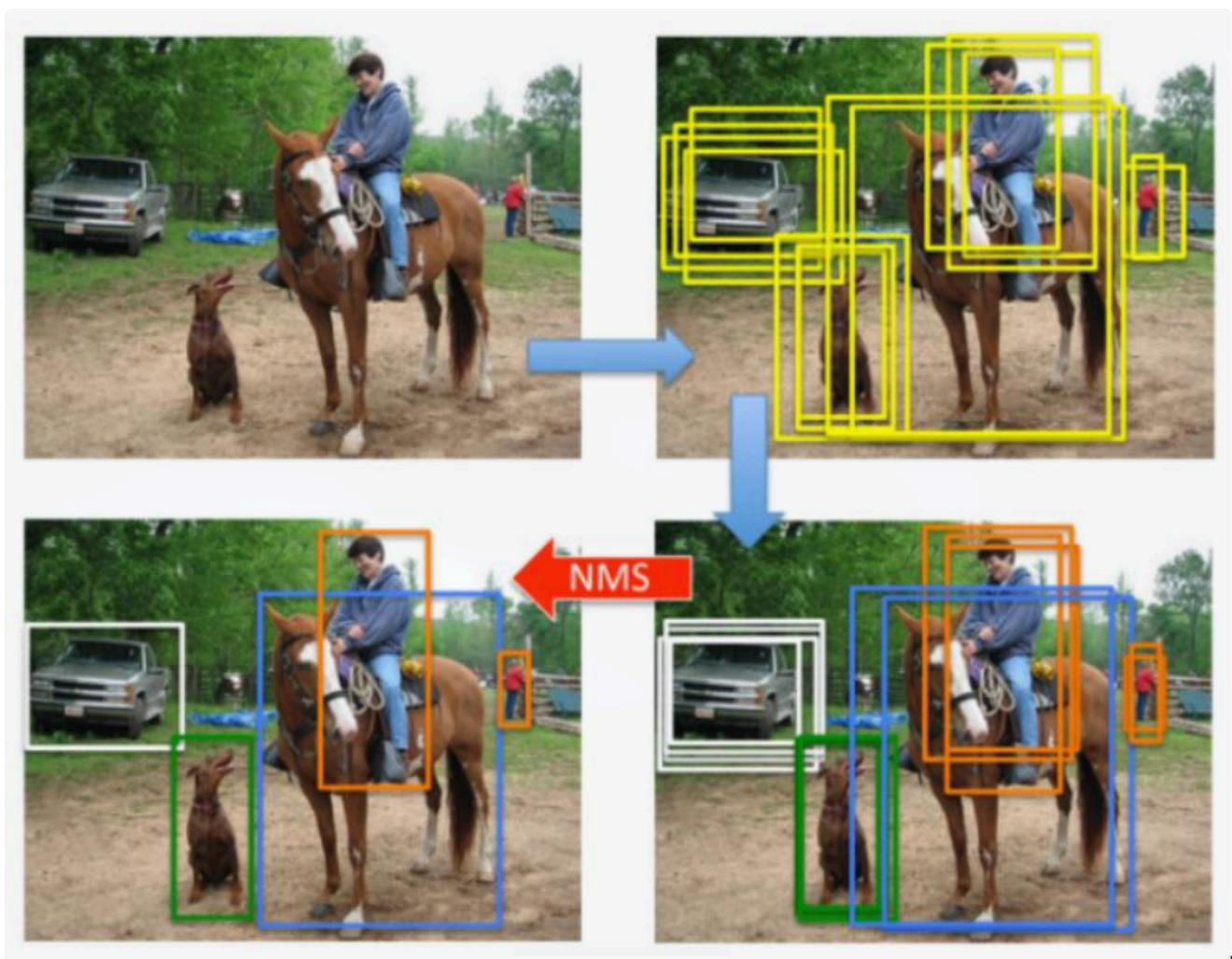
It is important to notice that, instead of directly predicting the new box coordinates, the regressor learns **four offset functions** d_x, d_y, d_w, d_h that describe how to **scale and shift the proposal box**.

It is important to notice that, since the **number of proposed regions** is **incredibly high**, a lot of them will be **redundant** and **overlapping** with **other bounding boxes**, which need to be **removed**.

To achieve this, we can employ **non-maximum suppression (NMS)**, a **greedy algorithm** working on one class at a time.

📋 NON-MAXIMUM SUPPRESSION

1. Among all boxes of a **particular class**, pick the **box with the highest confidence score** given by the SVM.
2. Use the chosen box as **ground truth** to compute the **IoU score** with **all other boxes of that class**.
3. **Remove** boxes with a **score higher than 70%**.
4. Choose the box with the **next highest confidence score** (of the same class) and **keep removing overlaps**. This is to account for **multiple instances** of the **same object** in the image.
5. **Repeat** steps 1 through 4 for **each object class**.



We said that the feature extraction step uses a pretrained CNN. This is true but not entirely accurate, as the chosen CNN **must be fine-tuned**: the weights are *initialised to their pretrained values*, and then they are *optimised for the specific task*.

🔧 FURTHER NOTES: CNN FINE-TUNING

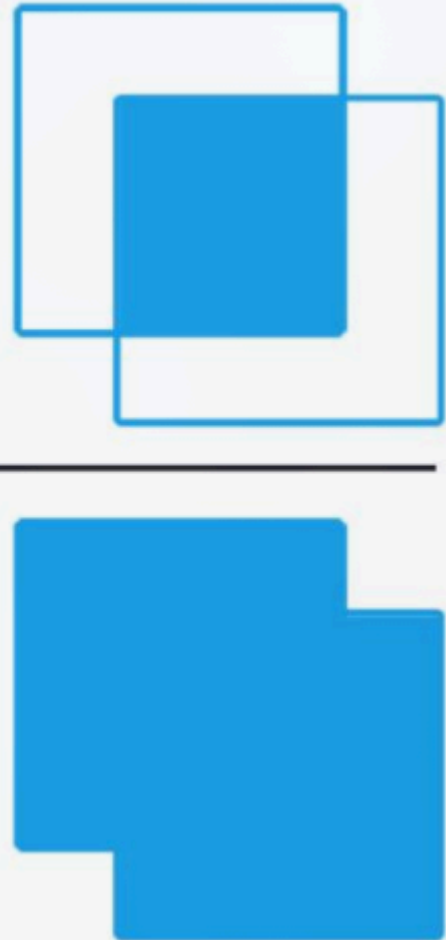
To fine tune the model, the **output layer** is **replaced** with one having an **extra neuron** to account for the **"background" class**, while the rest of the

architecture is left unchanged.

Then, *extra training steps* are performed with a **lower learning rate** (compared to the value used during pretraining): this is because we do **not want** the **weights to change much**.

To measure performance during fine-tuning, the **IoU** (*Intersection over Union*) metric is used, which measures **how close** the **predicted box** is to the **ground truth** ("label" box assigned to the training examples).

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



FAST R-CNN

As one may guess, performing **inference once per region proposal** is a **huge bottleneck** in R-CNNs.

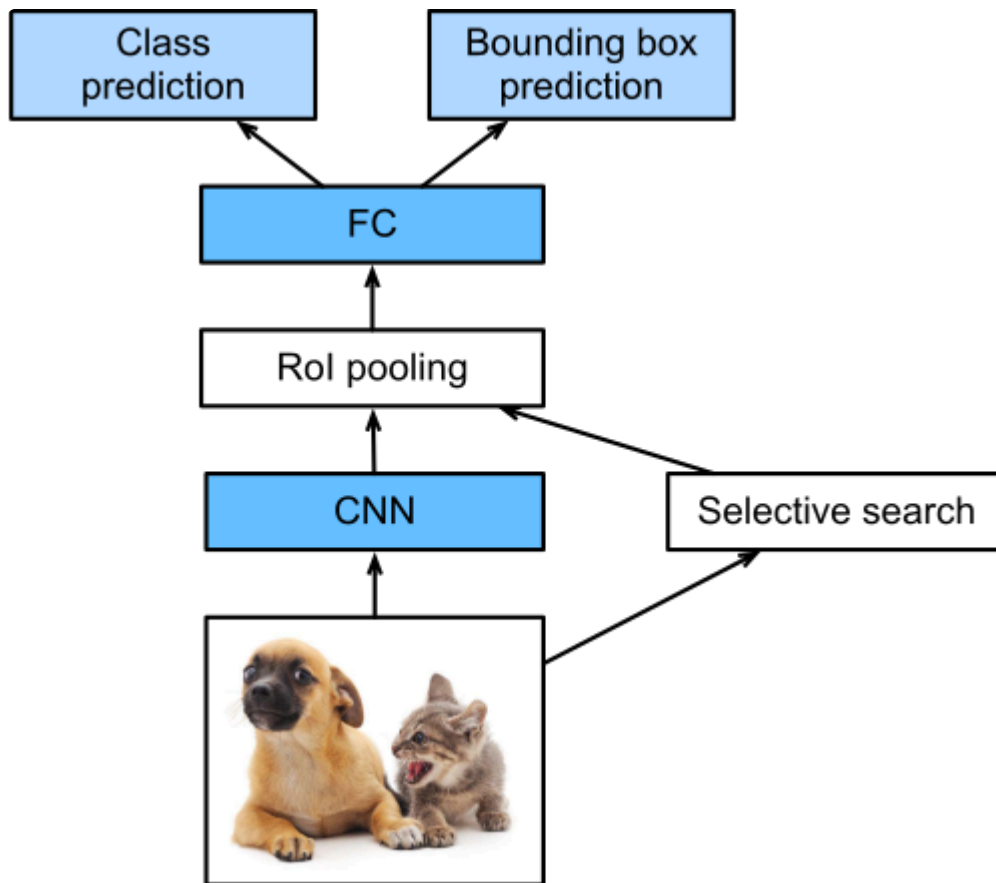
💡 R-CNN BOTTLENECK

Because the **region proposals** will inevitably **overlap**, performing forward propagation on each of them means **repeating the same calculations multiple times**, resulting in an **undesired computational complexity**.

This issue was solved with the introduction of the **fast R-CNN** model.

The **core idea** behind this model is to **run the CNN only once** on the **entire image**. The

figure below illustrates the fast R-CNN pipeline:



FAST R-CNN

1. Process the **entire image** with the CNN to produce a **feature map**.
Let the **shape** of this output be $1 \times c \times h_m \times w_m$.
2. Run **selective search** (independently of the CNN forward propagation step) to produce n **region proposals**; that is, n vectors of the form (x_i, y_i, h_i, w_i) .
3. **Map** the **region proposals** to the **feature map** to obtain **regions of interest (Rois)** of the form (x'_i, y'_i, h'_i, w'_i) . This mapping is necessary because selective search *operates on the original image*, but the *CNN output has a different shape*.
Once done, each RoI is "cut" from the feature map.
4. Use **RoI pooling** to bring **all the regions** down to the **same size** (e.g. h_r, w_r).
At this point, the **output tensor** has **shape** $n \times c \times h_r \times w_r$.
5. Feed this tensor to a **fully connected layer** to transform it into an **output** of **shape** $n \times d$, with $d = q + 4$, where q is the **number of classes**.

OUTPUT SIZE

For **each** of the n **Rois** we need to output q **confidence scores** to determine **which class** the object belongs to, and **4 numbers** to **identify its bounding box**.

As such, the output shape is $n \times (q + 4)$ as stated above.

As stated above, the **input** to the **FC layer** is a **concatenation of regions of interest**; we thus need a module to **transform** them so they all have the **same size** (so they can be concatenated in a tensor).

This module is the **RoI pooling layer**.

This **differs** from **standard pooling layers** because, while in the latter we *control the output shape by specifying sizes of the pooling window, padding and stride*, in the former we **directly specify the target output shape**.

For example, suppose we **specify the output height and width** for each region as h_r and w_r , respectively.

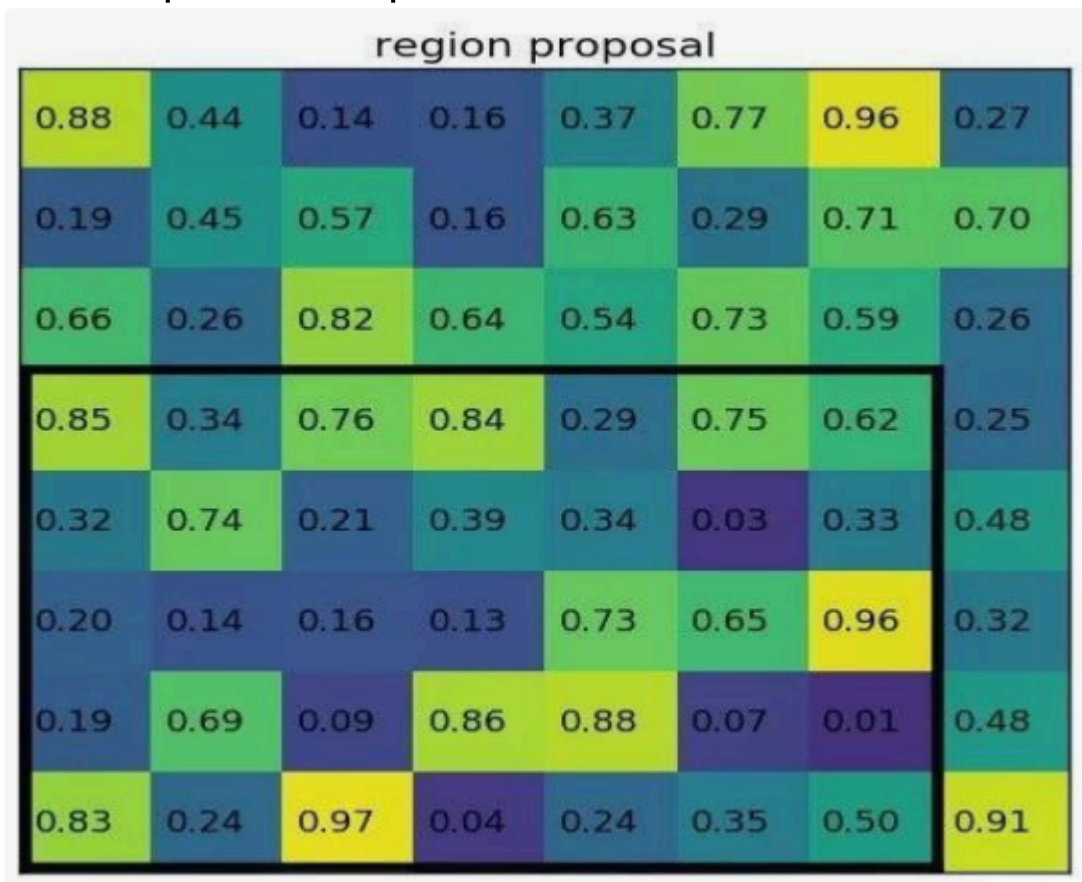
For **any region of interest** window of shape $h_i \times w_i$, this window is **divided** into an $h_r \times w_r$ **grid of subwindows**, where the shape of each subwindow is approximately given by $(h_i/h_r) \times (w_i/w_r)$.

This way, the RoI pooling layer can **extract features of the same shape** even when regions of interest have different shapes.

In practice, the **height and width** of any **subwindow** should be **rounded up**, and the **largest element** shall be used as the **output** of the subwindow (**max pooling**).

The figures below illustrate how RoI pooling works.

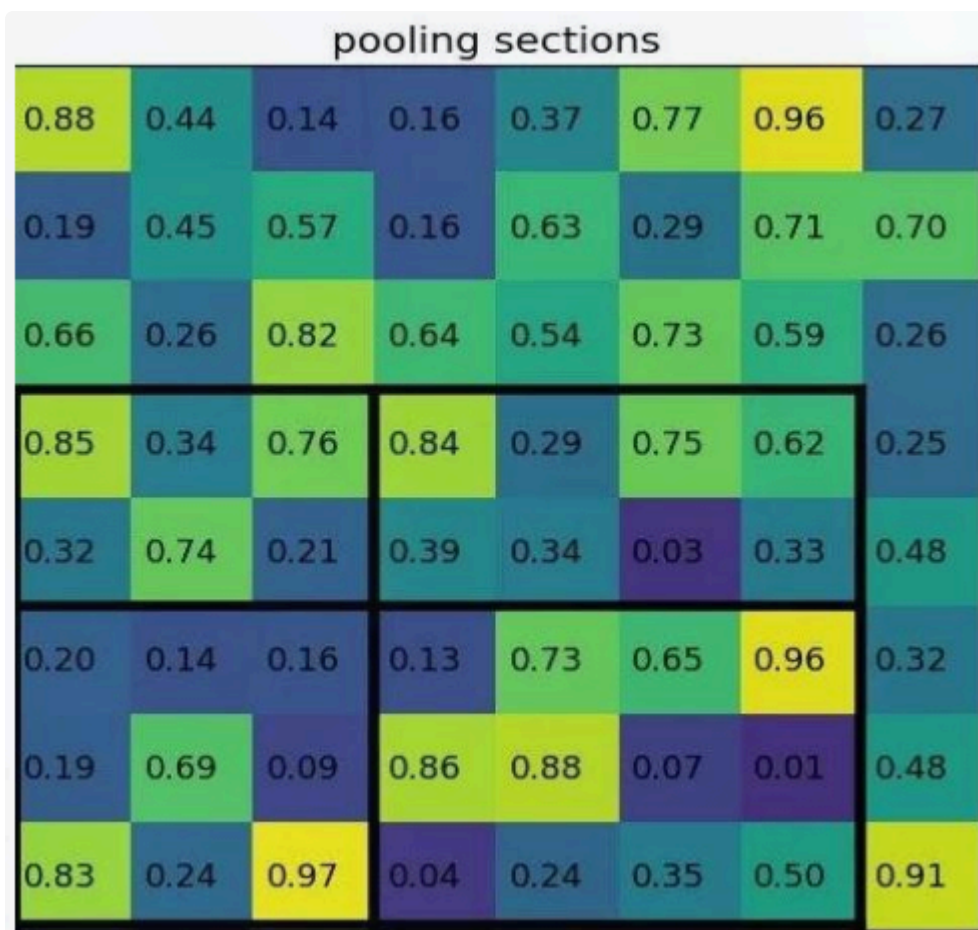
Suppose that **selective search** has generated the following **region proposal** on the CNN's output feature map:



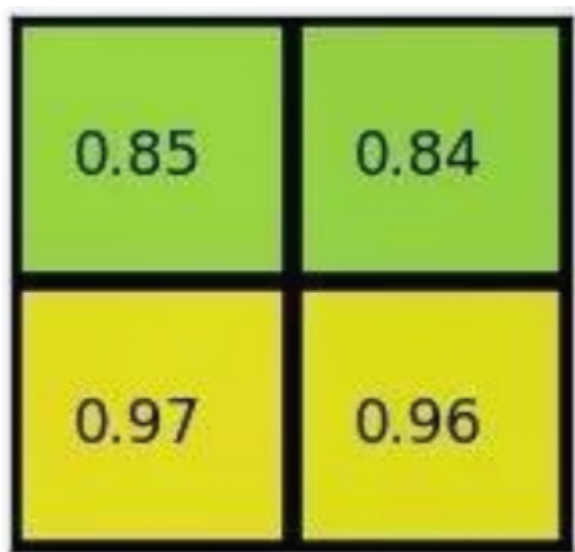
Now suppose we want our **output Rols** to be of **size 2x2**.

Notice that this size **doesn't perfectly divide** the **region proposal dimensions**.

As stated above, this is not a problem, as we can simply **round up** the **dimensions** of the **subwindows**:



As a result, we obtain the following **region of interest**:

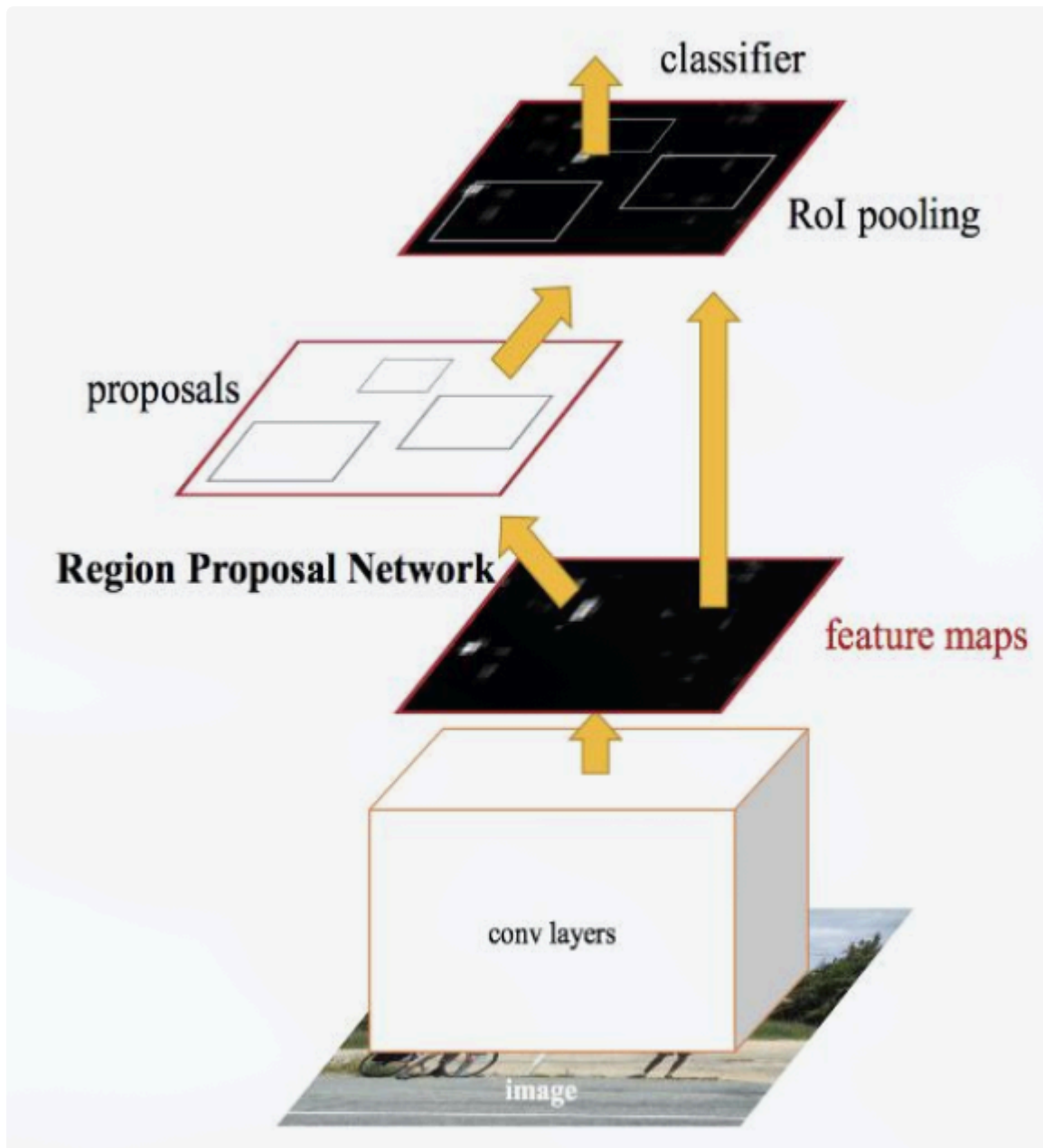


NOTE: BEYOND FAST R-CNN

The **R-CNN** family has been **improved** with new, **faster** models.

One such example is **Faster R-CNN**, which **speeds up** the **region proposal phase**.

To do so, it replaces *selective search* with a **dedicated region proposal network (RPN)**, which generates **less region proposals** without losing **accuracy**.



The figure above illustrates the **Faster R-CNN** pipeline. Notice that the **RPN** is **trainable** and can therefore (at least in some cases) achieve **better performance than selective search**.

YOLO

YOLO, acronym for **You Only Look Once** was the first **one-stage algorithm** to be developed.

By **unifying the phases of object detection** in a **single network**, YOLO is **significantly faster** compared to previous models; furthermore, YOLO is also **less likely** to incur in **false positives**

On the other hand, it is **more prone to localisation errors**.

It is important to notice that the **one-stage nature** of the algorithm doesn't *only* imply faster computation speeds. In fact:

ONE-STAGE IMPLICATIONS

Using a **single network** to perform **all phases** of the process **forces the model to reason over the entire image** at the same time rather than *processing individual regions*.

This results in a **better understanding of the environment** (*global context*) the objects are located in, and creates an **implicit link** between effectively **related classes** of objects.

Let us now look at how the YOLO algorithm approaches the problem; in particular, we will look at the **first implementation** of the model: **YOLO V1**.

YOLO V1

1. **Resize** and **divide** the original **image** into an $S \times S$ **grid** of cells. Each cell is **responsible for predicting a fixed number of boxes** B .
2. Use a **CNN** to **output a tensor** with a predefined set of n numbers **for each cell** in the grid.
3. Use **Non-Maximum Suppression** to **get rid of overlapping boxes**.

Now let us discuss each step in detail.

STEP 1

The input image is **resized** to the dimensions expected by the CNN (448×448 in the original paper) and then **divided** into a **grid** of $S \times S$ **cells** (the authors of the paper used $S = 7$, modern variants typically use $S = 19$).

Each cell is then required to **produce a fixed number** B of **bounding boxes** ($B = 2$ in the original paper) of the form $(\hat{x}_l, \hat{y}_l, \hat{h}_l, \hat{w}_l)$ with $l = 1, \dots, B$.

BOUNDING BOXES

- The numbers $\hat{x}_l, \hat{y}_l$ are the coordinates of the i -th **boundig box's centre**. They are **relative to the top-left corner** of the specific **cell** and **normalised between 0 and 1**: e.g. $(0.5, 0.5)$ is the centre of the cell and $(1, 1)$ is the cell's *bottom-right corner*.
- The numbers $\hat{h}_l, \hat{w}_l$ are the **height and width** of the i -th **bounding box**. These too are **normalised between 0 and 1**, but **relative to the entire**

image: e.g. (0.5, 0.5) means that *both the height and width of the bounding box are half of the original image dimensions.*

Furthermore, a **cell** is **responsible for the detection of an object** only if its **centre lies within the cell itself** (more context on this is provided further down the paragraph).

STEP 2

A **single forward pass** through the **CNN** is performed to generate an **output tensor** of **shape** $S \times S \times (B \cdot 5 + C)$ where C is the *number of possible classes*.

Because the original YOLO network used $S = 7$, $B = 2$ and $C = 20$, its output tensor shape was $7 \times 7 \times 30$.

🔍 OBSERVATION: OUTPUT TENSOR SHAPE

Each of the $S \times S$ cells outputs $n = B \cdot 5 + C$ number.

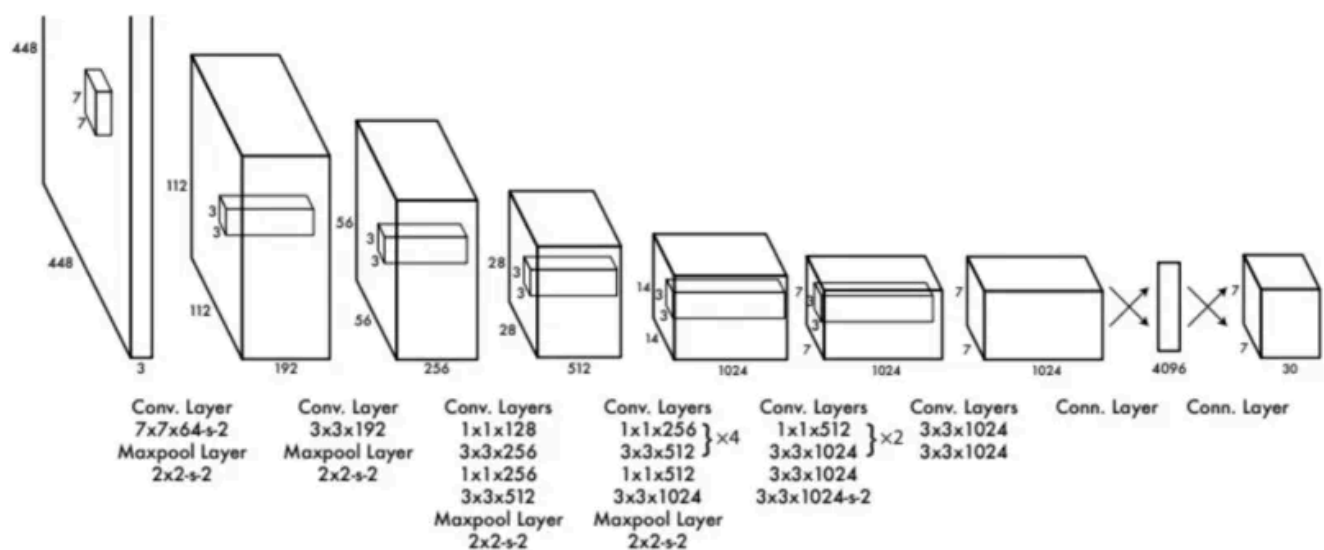
These are:

- **5 values for each box**; the first four identify the box $(\hat{x}_l, \hat{y}_l, \hat{h}_l, \hat{w}_l)$, and the fifth is a **confidence score** $\hat{c}_l$ which tells *how confident the model is* about the **presence of an object** in that particular **bounding box**.
- C **class-specific confidence scores** which tell *how confident the model is* that the **object in the grid belongs to each class**.

STEP 3

Because the **number of boxes** produced is **high**, **NMS** is required to get rid of **overlapping boxes**. The algorithm used is the same seen in > [REGION BASED CNN](#).

Finally, we can take a look at the overall architecture of the network, shown in the figure below:



Having established **how YOLO works**, let us now look at **how to train it**; that is, **how we need to construct a loss function** suitable for this **particular network**.

To start, it hopefully appears clear that our loss function needs to be comprised of **three terms**:

1. **Classification loss**, to account for *missclassifications*.
2. **Localisation loss**, to account for the *spatial error* between the *predicted box* and the *ground truth bounding box*.
3. **Confidence loss**, to account for the errors in the *confidence outputs*.

Notice that symbols with a *hat* denote the **model's predictions**, while symbols with *no hat* are the **target values**.

FIRST TERM: Classification Loss

$$\sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c=1}^C (p_i(c) - \hat{p}_i(c))^2$$

Here, $\mathbb{1}_i^{obj} = 1$ when an **object appears in the i -th cell** (0 otherwise).

In code, this can be implemented as a **tensor mask** and it enforces the **detection responsibility** introduced above; that is, **only** the cell **containing** the ground truth box's **centre** is **responsible for its detection**.

SECOND TERM: Localisation Loss

$$\begin{aligned} & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 - (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \end{aligned}$$

Here, the indicator variable $\mathbb{1}_{ij}^{obj} = 1$ if the j -th **bounding box** of the i -th **cell** is **responsible for recognising an object**; that is, if the **object's centre lies in the cell** *and* the j -th **box** (among those associated with cell i) is the one with the **higher IoU score**.

NOTE

The original paper states:

At training time we only want **one bounding box** predictor to be responsible **for each object**. We assign one predictor to be "responsible" for predicting an object based on which prediction has the **highest current IoU with the ground truth**.

Intuitively, this has the **effect of preventing bounding boxes from collapsing to the same shape**: if only one box is chosen for each object, the *others do not affect the gradients during backpropagation*, thus **maintaining their shape**, which may be **better at detecting other object classes**.

THIRD TERM: Confidence Loss

$$\sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_{ij} - \hat{C}_{ij})^2 + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_{ij} - \hat{C}_{ij})^2$$

Where the **indicator variable** is the **same** used for **localisation loss**.

Notice the the C_{ij} values here are the **confidence scores** of the **boxes**, not the *class-predictions*, which were used to compute the *classification loss*.

NOTE: WEIGHTING PARAMETER

The solution presented above would cause an **imbalance in the model**, as it **doesn't account** for the fact that **most boxes contain no objects** (only a fraction of all the proposed boxes actually contains an object, and even fewer are chosen to represent it).

To address this, we **must down-weight** the **contribution** of the **second term** by adding a $\lambda_{noobj} < 1$ parameter. in the original paper, $\lambda_{noobj} = 0.5$.

$$\lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_{ij} - \hat{C}_{ij})^2$$

Thus, the final loss function looks as follows:

$$\begin{aligned} L = & \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c=1}^C (p_i(c) - \hat{p}_i(c))^2 \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 - (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_{ij} - \hat{C}_{ij})^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_{ij} - \hat{C}_{ij})^2 \end{aligned}$$

BEYOND YOLO V1

Overall, YOLO V1 brought some **great benefits** when compared to **other solutions**: it was **fast**, it **eased training** and optimisation, and it was **more capable of generalisation**.

On the other hand, it also had some significant **drawbacks**:

DRAWBACKS

1. **Bad performance** when the image presents **groups of small objects**, because *each grid can only detect one object*.
2. High **localisation error** due to the fact that *bounding box ratios are totally learned from data*.

To address this issues, **YOLO V2** brought some **improvements** and **changes**, namely:

1. **Batch Normalisation**, which lead to significant improvements in convergence and *eliminated the need for other forms of regularisation*.
2. **High Resolution classifier**: by training the model on *high resolution images* for the *first 10 epochs*, the network could *adjust its filters* to work with *higher resolution inputs*.
3. **Anchor Boxes**: in YOLO V2, the **bounding boxes** are **predetermined** (using *K-means clustering* over the ground truth boxes in the training set) instead of being *learned by the network*. Furthermore, each cell produces **5 bounding boxes** instead of 2.
4. **Class prediction moved from cell to box**: in YOLO V2 each cell outputs $5 \cdot (5 + C)$ values, where C is the number of classes. That is, **every box can predict an object**: this solves the bad performance issue on images containing **groups of small objects** because **each cell can now predict more than one object**.
5. **New architecture**: the underlying CNN architecture was expanded with **more layers** (*DarkNet-19*, shown below).

Type	Filters	Size/Stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

It is important to notice that **YOLO V1** was released in **2015**, and **YOLO V2** in **2016**. Since then, the model family has **kept improving** (the figure below shows YOLO releases up until 2024):



YOLO is mostly known for its **object-detection** capabilities, however, the latest versions can tackle a multitude of computer vision tasks, and (as of Q3 2026) it's been announced that future models will expand into **3D perception**.

SEGMENTATION

Finally, it is time to discuss **segmentation**, which pushes computer vision to the **highest level of spatial granularity**: while **detection** answers "*what and where*" using rectangular bounding boxes, the goal of **segmentation** is to answer "*what*" at the level of **every single pixel** in the image.

This pixel-level precision is crucial for applications **where object boundaries are non-rectangular** (or in any application where a rectangular bounding box isn't precise

enough); these include **autonomous driving** (delineating drivable road surfaces), **medical imaging** (delineating tumors), and **photo editing** (precise cutouts).

Within segmentation, we distinguish two sub-tasks:

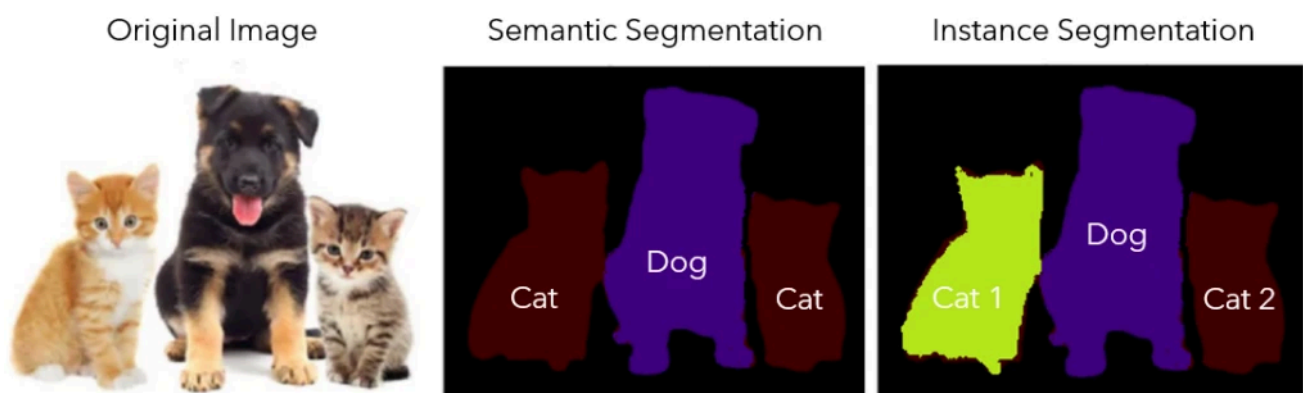
📖 SEMANTIC SEGMENTATION

The goal is to assign a **class label to every pixel** (e.g., "road", "sky", "car", "pedestrian"). Crucially, semantic segmentation does **not distinguish** between **different instances** of the **same class**; that is, all "car" pixels are treated identically, even if they belong to different vehicles.

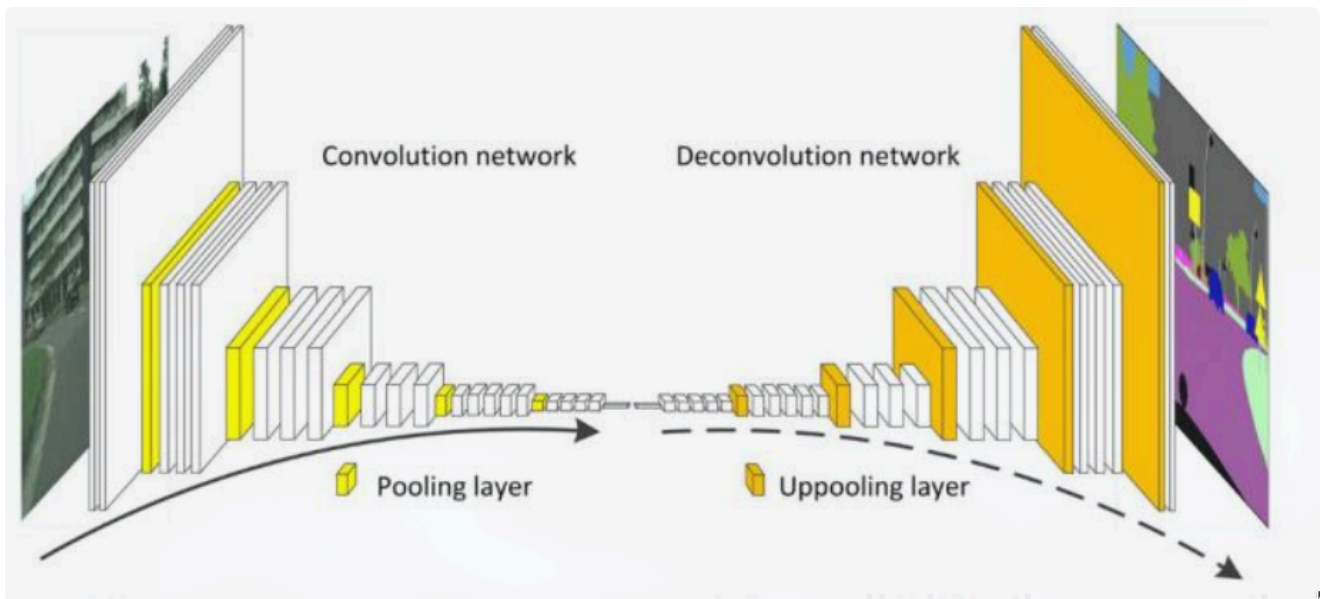
📖 INSTANCE SEGMENTATION

Also called **panoptic segmentation** when combined with semantic background labeling, this task goes a step further as it not only requires classifying each pixel but also **separating individual instances** of the same class (e.g., distinguishing "car #1" from "car #2").

This means **combining object detection** (instance localization) with **semantic segmentation** (per-pixel masks).



To achieve dense pixel-level predictions, standard CNNs (which reduce spatial resolution via pooling/strides) must be paired with **upsampling layers** (such as transposed convolutions) to restore the original image resolution, as illustrated below:



Notice that the output is a **tensor** of the **same size as the input** (in width and height, the number of channels is different), with a class (or instance) **label for every position**.

✍ DECONVOLUTION VS. TRANSPOSED CONVOLUTION

The **deconvolution network** shown above actually performs **transposed convolution** (sometimes misnamed deconvolution).

Unlike a true deconvolution (which mathematically inverts a convolution), transposed convolution is a **learnable upsampling operation** that **reconstructs spatial resolution**.

We will adopt the more precise term "transposed convolution" throughout the chapter to avoid confusion.

DEEPLAB V3+

DeepLabV3+ is a **semantic segmentation model** introduced by Google researchers in 2018 to **solve the loss of spatial information** suffered by previous architectures.

💡 SPATIAL INFORMATION LOSS

In traditional CNNs, performing operations like *max pooling* **reduce the image size to extract abstract features**.

However, **excessive shrinking** leads to the **loss of fine details** necessary for *accurate* segmentation.

DeepLabV3+ addresses this by introducing a novel **Encoder-Decoder architecture** that captures rich semantic information while recovering sharp object boundaries.

Before diving into the details of V3+, we must briefly look at its predecessors.

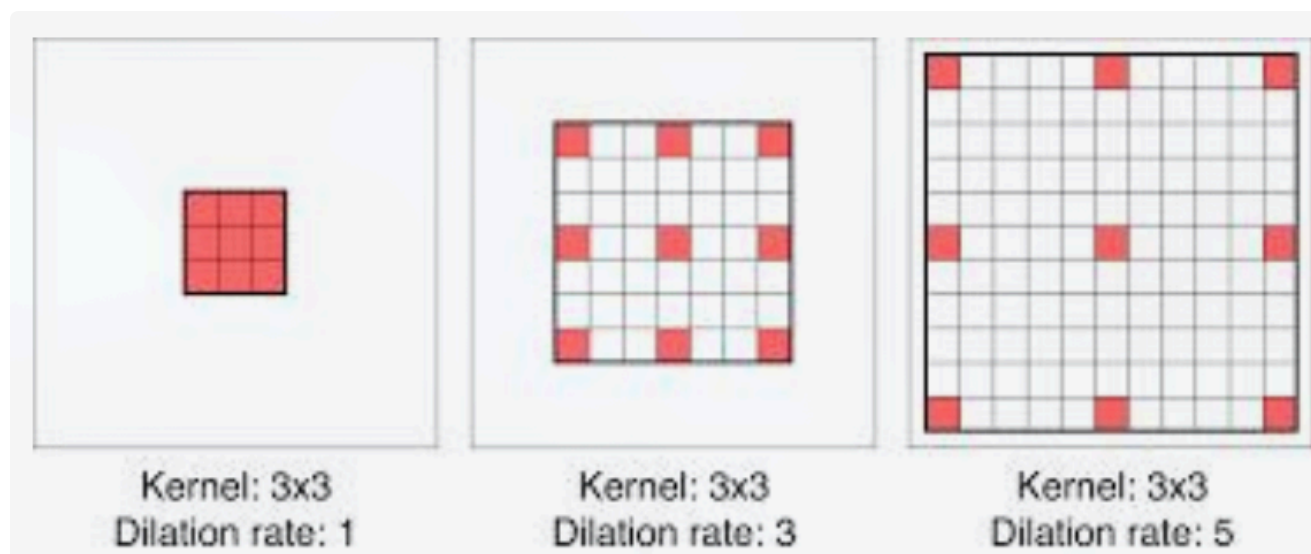
DeepLab V1 and V2 introduced **Atrous (Dilated) Convolutions** to **control the resolution of features**.

For each location (i, j) on the *output feature map* $\mathbf{y}$ and a *filter* $\mathbf{w}$, an **atrous convolution** of *rate* r is applied over the *input feature map* $\mathbf{x}$ as follows:

📖 ATROUS CONVOLUTION

$$\mathbf{y}[i, j] = \sum_k \sum_l \mathbf{x}[i + rk, j + rl] \cdot \mathbf{w}[k, l]$$

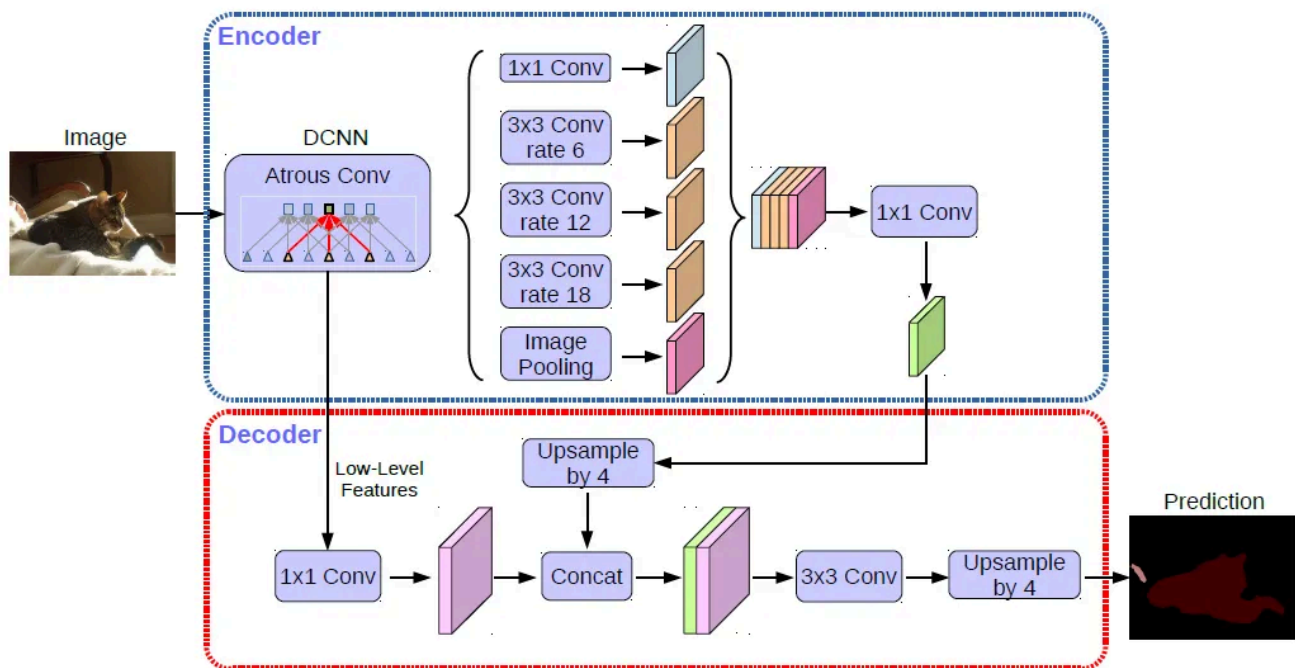
Where the **rate** r determines the stride with which we sample the input feature. Note that the *standard convolution* is a special case in which $r = 1$.



Thus, the **filter's field of view** is **adaptively modified** by changing the rate value.

DeepLab V3 then brought improvements and refinements over previous models. Finally, **DeepLab V3+** was developed after researchers realized that, while V3 was *great at understanding "what" is in the image*, it *struggled with "where" the edges were exactly*. V3+ adds a **decoder module** to **refine the segmentation results**, resulting in much **sharper boundaries** compared to V3.

The DeepLabV3+ architecture is shown in the figure below:



As introduced above, the network is made up of *two main components*: an **encoder** and a **decoder**.

The **encoder** is essentially the DeepLabV3 network, which is itself made up of:

- A **backbone network** (*feature extractor*). DeepLabV3+ usually employs networks like ResNet-101 for its backbone, with the *key modification* that **atrous convolutions** are used in place of standard convolutions.
- An **ASPP** (*Atrous Spatial Pyramid Pooling*) **module**, responsible for **handling scale variations** in the **objects** in the **input images** (e.g. due to their distance from the camera). This is achieved by applying **multiple atrous convolutions** with **different rates** simultaneously.

While the **previous model** DeepLabV3 *simply upsampled the output* by a factor of 16, resulting in **rough edges**, DeepLabV3+ **solves** this problem by using a **feature-fusion strategy** in the **decoder** module.

To understand **how the decoder works**, consider the explanation below.

📋 DECODER MODULE

- First, the **high-level features** coming from the encoder (the ASPP output) are **upsampled** by a factor of 4 using bilinear interpolation.
- At the same time, the model takes **low-level features from earlier backbone layers**, which contain **strong spatial details** (like edges and boundaries) but have **weaker semantic information**.
- Because low-level features usually have **many channels**, a 1×1 convolution is applied to **reduce their depth** so they do not dominate the high-level features.

- After this, the upsampled high-level features and the channel-reduced low-level features are **concatenated**.
- Then a few 3×3 convolutions **refine the combined representation**.
- In the final step, the output is upsampled again by a factor of 4 to return to the **original image resolution**.

LOSS FUNCTIONS FOR SEGMENTATION

Segmentation models like DeepLabV3+ usually employ **pixel-wise cross entropy loss** during training.

However, **depending on the dataset**, it might be **better** to use **other kinds of loss functions** to account for class imbalances (e.g. small objects vs large background)

One such loss function is the **generalised dice loss** (derived from the Sorensen-Dice coefficient).

📖 SORENSEN-DICE COEFFICIENT

$$\text{Dice} = \frac{2 \times \text{TP}}{(\text{TP} + \text{FP}) + (\text{TP} + \text{FN})} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Notice that this coefficient takes values in the range $[0, 1]$; in particular, $\text{Dice} = 0$ when **all the inputs are wrongly classified** and $\text{Dice} = 1$ when all the inputs are correctly classified.

Because during training we want to **minimise the loss**, the **dice loss** is simply the **inverse of the dice coefficient**; that is:

$$\text{DL} = 1 - \text{Dice}$$

This is then **generalised** to the **multi-class** case as follows:

📖 GENERALISED DICE LOSS

Given **predictions** Y and **targets** T , **pixels** $m = 1, \dots, M$, **classes** $k = 1, \dots, K$, the **generalised dice loss** is defined as:

$$\text{GDL}(Y, T) = 1 - \frac{2 \sum_{k=1}^K w_k \sum_{m=1}^M Y_{km} T_{km}}{\sum_{k=1}^K w_k \sum_{m=1}^M Y_{km} + T_{km}}$$

Where $T_{km} = 1$ if pixel m belongs to class k and the **factors** w_k are given by:

$$w_k = \frac{1}{\left(\sum_{m=1}^M T_{km}\right)^2}$$

WEIGHT FACTORS

The weight factors w_k are **inversely proportional** to the **frequency of class k** . This guarantees that the network **evaluates small classes** (whose labels appear with lower frequency) **fairly**.

Notice that GDL **treats false positives and false negatives equally**, which is **not always ideal**.

Think of **tumor diagnoses** for example: a **false negative** (missing the tumor) is **much worse** than a false positive (falsely detecting one).

To address this issue, the **Tversky index** generalises the **dice coefficient** by introducing **two weighting factors**:

TVERSKY INDEX

$$\text{TI}_k = \frac{\text{TP}}{\text{TP} + \alpha \text{FN} + \beta \text{FP}}$$

Notice that when $\alpha = \beta = 0.5$ we have $\text{TI}_k = \text{Dice}_k$.

The **Tversky Loss** is then computed by **adding the inverse Tversky indices of all classes**:

TVERSKY LOSS

$$\text{TL} = \sum_{k=1}^K (1 - \text{TI}_k)$$